Trabalho – Geração do Conjunto de Mandelbrot usando Pthreads

1. Objetivo

Implementar dois programas para calcular o **conjunto de Mandelbrot**, um sequencial e outro paralelo utilizando a biblioteca **Pthreads**, e avaliar o desempenho da execução paralela.

Na versão paralela, o cálculo deverá ser dividido entre os threads, de forma que cada thread seja responsável pelo processamento de um subconjunto dos pixels da imagem.

Ao final, deverão ser comparados os tempos de execução das versões sequencial e paralela e calculado o **speedup** obtido com diferentes números de threads.

Os experimentos de desempenho deverão ser realizados no **Cluster do IME**, disponível no **Laboratório de Computação de Alto Desempenho para Defesa Cibernética – LCAD2C**.

---

2. Conjunto de Mandelbrot

O conjunto de Mandelbrot é definido a partir da sequência:

$$z_0 = 0$$

$$z_{n+1} = z_n^2 + c$$

onde z e c são números complexos.

Para cada ponto c do plano complexo, a sequência é calculada até que ocorra uma das seguintes condições:

$$|z_n| > 2$$

ou seja atingido um número máximo de iterações.

Neste trabalho, cada ponto do plano complexo será associado a um pixel de uma imagem. Para cada pixel deverá ser armazenado o número de iterações realizadas até que o ponto ultrapasse o limite estabelecido ou seja atingido o número máximo de iterações.

---

3. Algoritmo Sequencial

Considere uma imagem com `altura` linhas e `largura` colunas. Cada pixel `(i,j)` representa um ponto do plano complexo.

O algoritmo sequencial a ser utilizado como base para o trabalho é apresentado a seguir:

```
for (i = 0; i < altura; i++) {
    for (j = 0; j < largura; j++) {

        /* Converte o pixel (i,j) em um ponto do plano complexo */
        cx = xmin + j * (xmax - xmin) / (largura - 1);
        cy = ymin + i * (ymax - ymin) / (altura - 1);

        zx = 0.0;
        zy = 0.0;
        iter = 0;

        while ((zx*zx + zy*zy <= 4.0) && (iter < MAX_ITER)) {

            novo_zx = zx*zx - zy*zy + cx;
            novo_zy = 2.0*zx*zy + cy;

            zx = novo_zx;
            zy = novo_zy;

            iter++;
        }

        imagem[i][j] = iter;
    }
}
```

Para todos os experimentos deverão ser utilizados os seguintes limites para o plano complexo:

```
xmin = -2.0;
xmax =  1.0;
ymin = -1.5;
ymax =  1.5;

MAX_ITER = 1000;
```

O valor armazenado em `imagem[i][j]` corresponde ao número de iterações realizadas para o ponto associado ao pixel `(i,j)`.

---

4. Implementação Paralela

A partir do algoritmo sequencial apresentado, deverá ser implementada uma versão paralela utilizando a biblioteca **Pthreads**.

O cálculo dos pixels deverá ser distribuído entre os threads de modo que:

- todos os pixels da imagem sejam calculados;
- cada pixel seja calculado por apenas um thread;

- o trabalho de cálculo seja distribuído entre os threads.

A estratégia utilizada para dividir os pixels entre os threads deverá ser definida pelo aluno.

O programa deverá receber como parâmetro o número de threads a serem utilizadas.

O thread principal deverá criar os threads de trabalho utilizando `pthread_create()` e aguardar o término de todas elas utilizando `pthread_join()`.

Os resultados calculados pela versão paralela deverão ser os mesmos obtidos pela versão sequencial.

---

5. Ambiente de Execução

Os experimentos para avaliação de desempenho deverão ser realizados **obrigatoriamente no Cluster do IME**, disponível no **Laboratório de Computação de Alto Desempenho para Defesa Cibernética – LCAD2C**.

As informações sobre o laboratório estão disponíveis em:

https://sites.google.com/ime.eb.br/lcad2c/p%C3%A1gina-inicial

As versões sequencial e paralela utilizadas para obtenção dos tempos de execução deverão ser compiladas e executadas no cluster.

Antes da realização dos experimentos, o grupo deverá identificar as principais características do nó utilizado, incluindo:

- modelo do processador;
- número de processadores físicos;
- número de núcleos disponíveis;
- quantidade de memória principal.

Essas informações deverão ser apresentadas no relatório.

---

6. Experimentos

Inicialmente, execute a versão sequencial e obtenha seu tempo de execução.

Em seguida, execute a versão paralela utilizando os seguintes números de threads:

$$2,\ 4,\ 8,\ 16, 32$$

Os experimentos deste trabalho estão limitados a **32 threads**, mesmo que o nó utilizado possua um número maior de processadores disponíveis.

Essa limitação deverá ser mantida por todos os alunos, de forma a padronizar os experimentos e reduzir o tempo total de utilização do cluster.

Os experimentos deverão ser realizados para os seguintes tamanhos de imagem:

- 1.000 × 1.000 pixels;
- 2.000 × 2.000 pixels;
- 4.000 × 4.000 pixels;
- 8.000 × 8.000 pixels.

O valor de `MAX_ITER` deverá ser mantido em **1000** em todos os experimentos.

Para cada combinação entre tamanho da imagem e número de threads, o programa deverá ser executado **pelo menos três vezes**. O tempo considerado na análise deverá ser a média dos tempos obtidos.

Assim, para cada tamanho de imagem deverão ser obtidos resultados para:

p = 1, 2, 4, 8, 16, 32

threads.

O tempo medido deverá corresponder apenas ao processamento necessário para calcular o conjunto de Mandelbrot. O tempo gasto para inicialização de estruturas de dados ou eventual gravação dos resultados em arquivo não deverá fazer parte da medição.

Todos os tempos utilizados nos gráficos e no cálculo do speedup deverão ser obtidos no Cluster do IME.

---

7. Análise de Desempenho

Para cada tamanho de problema, deverá ser calculado o speedup obtido utilizando p threads:

$$S(p) = \frac{T_{seq}}{T_p}$$

onde:

- T_{seq} é o tempo médio de execução do programa sequencial no cluster;
- T_p é o tempo médio de execução do programa paralelo utilizando p threads no cluster.

O relatório deverá apresentar os seguintes gráficos:

7.1. Tempo de execução × número de threads

Apresentar o tempo médio de execução do programa paralelo à medida que o número de threads aumenta.

O eixo X deverá representar:

$$1,\ 2,\ 4,\ 8,\ 16,\ 32$$

threads.

Deverá ser apresentada uma curva para cada tamanho de imagem utilizado.

7.2. Speedup × número de threads

Apresentar o speedup obtido à medida que o número de threads aumenta.

O eixo X deverá representar o número de threads e o eixo Y deverá representar o speedup. Deverá ser apresentada uma curva para cada tamanho de imagem.

---

8. Discussão dos Resultados

Com base nos experimentos realizados, responda às seguintes questões:

a) O aumento do número de threads sempre provocou uma redução do tempo de execução? Justifique os resultados observados.

b) Compare o speedup obtido experimentalmente com o speedup ideal. Considerando p threads, o speedup ideal seria:

$$S(p) = p$$

Os valores obtidos ficaram próximos desse comportamento? Explique.

c) O tamanho da imagem teve influência no speedup obtido? Compare os resultados para os diferentes tamanhos de problema e explique o comportamento observado.

d) Ao aumentar o número de threads de 1 até 32, houve algum ponto a partir do qual o ganho de desempenho se tornou pequeno? Caso isso tenha ocorrido, apresente possíveis razões para esse comportamento.

---

9. Entrega

Deverão ser entregues:

- código-fonte do programa sequencial;
- código-fonte do programa paralelo utilizando Pthreads;
- relatório contendo:
    - descrição da implementação realizada;
    - descrição da estratégia utilizada para distribuir o trabalho entre as threads;
    - descrição do nó do Cluster do IME utilizado nos experimentos;
    - tabelas contendo os tempos médios de execução;
    - gráficos de tempo de execução e speedup;
    - respostas às questões propostas na seção de discussão dos resultados.

Todos os resultados de desempenho apresentados no relatório deverão ter sido obtidos no **Cluster do IME do Laboratório de Computação de Alto Desempenho para Defesa Cibernética – LCAD2C**.